Todos los cambios, explicados

Qué cambiaste en tu app de clima para que muestre Pokémon con PokeAPI, y **por qué** hizo falta cada cambio.

La idea general

Tu app está dividida en capas. La pantalla solo sabe mostrar **entidades** (`RespuestaClima`, `Ubicacion`, `ClimaActual`…) y no le importa de dónde vienen los datos. Por eso basta con cambiar la capa que **trae** los datos (`datos/`) y hacer que siga entregando las mismas entidades.

```
Buscador → ClimaRepository → WeatherApiService → ApiClient → Internet (PokeAPI)
                                  (cambiado)        (cambiado)
Pantalla ← Entidades ← ClimaMapper ← ClimaDtos ← JSON del Pokémon
          (iguales)   (cambiado)    (cambiado)
```

#	Archivo	Qué cambió	Por qué, en una frase
1	`apikey.properties`	Archivo nuevo con una clave cualquiera	El código exige una clave, aunque PokeAPI no use ninguna
2	`ApiClient.kt`	La URL base	Ahora nos conectamos a otro servidor
3	`WeatherApiService.kt`	La ruta y cómo viajan los datos	PokeAPI tiene otra dirección y rechaza los parámetros de WeatherAPI
4	`ClimaDtos.kt`	`RespuestaClimaDto` + nueva `SpritesDto`	El JSON del Pokémon tiene otra forma
5	`ClimaMapper.kt`	La última función	Hay que acomodar los datos del Pokémon en las casillas de clima

No cambiaron: `ClimaRepository`, `ClimaViewModel`, `ClimaApp` (pantallas), las entidades y las demás clases y funciones de los DTOs y del mapper. **Por qué:** mantuvimos los mismos nombres (`obtenerPronostico`, `RespuestaClimaDto`, `aEntidad`), así que el resto del código no se entera del cambio.

Cambio 1 · apikey.properties (archivo nuevo)

`Api-main/apikey.properties`

```
WEATHER_API_KEY=pokeapi
```

¿Por qué?

PokeAPI **no necesita clave**, pero tu código fue hecho para WeatherAPI y la exige en dos lugares:

- `ClimaRepository.kt`: si la clave está vacía, muestra "*Falta la clave de WeatherAPI*" y no consulta nada.
- `ClimaViewModel.kt`: si la clave está vacía, enciende el **modo demo** y solo muestra datos falsos de La Paz.

Para no tocar esos archivos, le damos un valor cualquiera. Gradle lee este archivo al compilar y crea `BuildConfig.WEATHER_API_KEY = "pokeapi"`.

¿Por qué en la raíz y no dentro de `java/` ?

No es código, es un archivo de **configuración** (`NOMBRE=valor`). `build.gradle.kts` lo busca en la carpeta raíz del proyecto (`Api-main`). Si lo creas como clase Java, no compila.

Cambio 2 · ApiClient.kt: la URL base

`antes`

```
.baseUrl("https://api.weatherapi.com/v1/")
```

`después`

```
.baseUrl("https://pokeapi.co/api/v2/")
```

¿Por qué?

`ApiClient` decide **a qué servidor** se conecta la app. La URL base es la parte que se repite en todas las peticiones. La sacamos de la URL de ejemplo:

`https://pokeapi.co/api/v2/` + `pokemon/ditto` → lo de antes de `pokemon/` es la base.

Termina en `/` porque Retrofit lo exige. Si no, la app se cierra al abrir con "*baseUrl must end in /*".

Cambio 3 · WeatherApiService.kt: qué se pide y cómo

antes

```
import retrofit2.http.GET
import retrofit2.http.Query

interface WeatherApiService {
    @GET("forecast.json")
    suspend fun obtenerPronostico(
        @Query("key") key: String,
        @Query("q") ciudad: String,
        @Query("days") dias: Int = 3,
        @Query("lang") idioma: String = "es",
        @Query("aqi") calidadAire: String = "no",
        @Query("alerts") alertas: String = "no"
    ): RespuestaClimaDto
}
```

después

```
import retrofit2.http.GET
import retrofit2.http.Path
import retrofit2.http.Header

interface WeatherApiService {
    @GET("pokemon/{nombre}")
    suspend fun obtenerPronostico(
        @Header("key") key: String,
        @Path("nombre") ciudad: String,
        @Header("days") dias: Int = 3
    ): RespuestaClimaDto
}
```

3.1 · `@GET("forecast.json")` → `@GET("pokemon/{nombre}")`

¿Por qué?

`@GET` es la **ruta** que va después de la URL base. Comparando URLs se ve qué parte es fija y qué parte cambia:

`pokemon/ditto`, `pokemon/pikachu`, `pokemon/25` → `pokemon/` es fijo y el nombre cambia.

Lo que cambia se escribe como un **hueco** entre llaves: `{nombre}`. Retrofit lo rellena con lo que el usuario escribe en el buscador.

3.2 · `@Query("q") ciudad` → `@Path("nombre") ciudad`

¿Por qué?

En WeatherAPI la ciudad iba **después del** `?`: `forecast.json?q=La Paz`. Eso es `@Query`.

En PokeAPI el nombre va **dentro de la ruta**: `pokemon/ditto`. Eso es `@Path`.

El texto de `@Path("nombre")` tiene que ser **igual** al de las llaves `{nombre}`; así Retrofit sabe qué hueco rellenar.

La variable sigue llamándose `ciudad` a propósito, para que `ClimaRepository` no tenga que cambiar.

3.3 · `key` y `dias`: `@Query` → `@Header`

> **¿Por qué?**
> **PokeAPI rechaza los parámetros que no conoce**: `pokemon/ditto` → 200 ✅, pero `pokemon/ditto?days=3` → **403** ❌. Con `@Query`, la app mostraba "*El servicio no pudo responder (HTTP 403)*".
> Pero `key` y `dias` **no se pueden borrar**, porque `ClimaRepository` llama a la función así: `api.obtenerPronostico(key, consulta, 3)` ← siempre pasa la clave y el 3.
> `@Header` manda el dato **dentro de la petición**, no en la URL. PokeAPI lo ignora sin dar error. Así el repositorio sigue funcionando sin cambios.

3.4 · Se borraron `lang`, `aqi` y `alerts`

> **¿Por qué?**
> Por la misma razón (PokeAPI daría 403). Estos sí se pueden borrar porque tienen un valor por defecto (`= "es"`, `= "no"`) y `ClimaRepository` **nunca los pasa**. Nadie los echa de menos.

3.5 · Los imports

> **¿Por qué?**
> Kotlin solo conoce `@Path` y `@Header` si los importas. Sin el import salen en rojo: "*Unresolved reference: Path*". Y `import retrofit2.http.Query` ya no se usa (sale en gris): se puede borrar.

Cambio 4 · ClimaDtos.kt: la forma del JSON

antes

```
data class RespuestaClimaDto(
    @SerializedName("location") val ubicacion: UbicacionDto,
    @SerializedName("current") val actual: ClimaActualDto,
    @SerializedName("forecast") val pronostico: PronosticoDto
)
```

después

```
data class RespuestaClimaDto(
    @SerializedName("id") val id: Int,
    @SerializedName("name") val nombre: String,
    @SerializedName("height") val altura: Int,
    @SerializedName("weight") val peso: Int,
    @SerializedName("sprites") val sprites: SpritesDto
)

data class SpritesDto(
    @SerializedName("front_default") val imagen: String?
)
```

4.1 · ¿Por qué se cambia `RespuestaClimaDto`?

¿Por qué?

Un DTO **copia la forma del JSON**. Gson lee el JSON y llena cada `val` buscando el nombre que va en `@SerializedName("...")`.

El JSON de WeatherAPI tenía tres objetos: `"location"`, `"current"` y `"forecast"`. **El JSON del Pokémon no tiene ninguno**: tiene `"id"`, `"name"`, `"height"`... sueltos en el primer nivel.

`RespuestaClimaDto` es la clase que recibe el JSON completo (es lo que devuelve `obtenerPronostico`), así que ahí van los campos del Pokémon.

4.2 · ¿Por qué esos tipos?

En el JSON	Valor	Tipo	Por qué
`"id"`	`132`	`Int`	Número sin decimales
`"name"`	`"ditto"`	`String`	Lleva comillas: es texto
`"height"`	`3`	`Int`	Número sin decimales (en decímetros)
`"weight"`	`40`	`Int`	Número sin decimales (en hectogramos)
`"sprites"`	`{ ... }`	`SpritesDto`	Lleva llaves: es otro objeto y necesita su propia clase

4.3 · ¿Por qué hace falta `SpritesDto`?

> **¿Por qué?**
> En el JSON, `"sprites"` no es un texto ni un número, sino un objeto: `"sprites": { "front_default": "https://...png" }`. **Llaves = una clase nueva**. Es el mismo caso que `CondicionDto`, que existe porque `"condition"` también llevaba llaves.
> • `data class` con **paréntesis** `( )`: los `val` van en el constructor y Gson los llena. Con llaves `{ }` da error.
> • `String?` lleva `?` porque algunos Pokémon no tienen imagen y viene `null`. Sin el `?`, la app podría fallar.

4.4 · ¿Por qué no se tocan las otras clases?

> **¿Por qué?**
> `CondicionDto`, `UbicacionDto`, `ClimaActualDto`, etc. ya no se usan para leer el JSON, pero las funciones de arriba del mapper sí las usan. Si las cambias, esas funciones dan error ("*Type mismatch*"). Si las dejas igual, todo compila.

Cambio 5 · ClimaMapper.kt: acomodar los datos en las casillas

antes (la última función)

```
fun RespuestaClimaDto.aEntidad(): RespuestaClima = RespuestaClima(
    location = ubicacion.aEntidad(),
    current = actual.aEntidad(),
    forecast = pronostico.aEntidad()
)
```

después

```
fun RespuestaClimaDto.aEntidad(): RespuestaClima = RespuestaClima(
    location = Ubicacion(
        name = nombre,
        region = "Pokémon #$id",
        country = "",
        localtime = "",
        lat = altura / 10.0,
        lon = peso / 10.0
    ),
    current = ClimaActual(
        temp_c = altura / 10.0,
        feelslike_c = peso / 10.0,
        humidity = 0,
        wind_kph = 0.0,
        precip_mm = 0.0,
        uv = 0.0,
        last_updated = "",
        condition = Condicion(
            text = nombre,
            icon = sprites.imagen.orEmpty(),
            code = id
        )
    ),
    forecast = Pronostico(forecastday = emptyList())
)
```

¿Por qué cambia?

La versión de antes usaba `ubicacion`, `actual` y `pronostico`, que eran los `val` del `RespuestaClimaDto` viejo. Al cambiar el DTO (Cambio 4) ya no existen, y por eso salían en rojo.

Ahora **armamos cada entidad directamente**, como quien llena un formulario: a la izquierda va **la casilla** (copiada de las funciones de arriba) y a la derecha **el dato del Pokémon** que va en ella.

¿Por qué cada línea?

Línea	Se ve en pantalla como	Por qué así
`name = nombre`	Título grande	Ambos son `String`: pasa directo
`region = "Pokémon #$id"`	Subtítulo	`region` pide texto y `id` es número. Con `$` se mete dentro de un texto
`country = ""`, `localtime = ""`	Vacío	No hay dato del Pokémon para eso. Texto vacío = `""`
`lat = altura / 10.0`	Detalles: Latitud	Decímetros → metros (3 → 0,3). El `.0` lo convierte en `Double`, que es lo que pide `lat`
`lon = peso / 10.0`	Detalles: Longitud	Hectogramos → kg (40 → 4,0)
`temp_c = altura / 10.0`	El número grande	Elegiste mostrar la altura en el lugar más visible
`feelslike_c = peso / 10.0`	"Sensación térmica de..."	Elegiste mostrar el peso ahí
`humidity = 0`, `wind_kph = 0.0`...	0	Sin dato. `0` si la casilla es `Int`, `0.0` si es `Double`
`icon = sprites.imagen.orEmpty()`	La imagen (donde iba el ícono del clima)	`imagen` es `String?` y `icon` pide `String`. `.orEmpty()` cambia `null` por `""`
`code = id`	(no se ve)	Ambos son `Int`
`forecast = Pronostico(forecastday = emptyList())`	Pestaña Pronóstico vacía	Un Pokémon no tiene pronóstico de 3 días: lista vacía

¿Por qué no se tocan las otras 6 funciones del mapper?
Usan las clases viejas (`CondicionDto`, `UbicacionDto`...), que siguen existiendo sin cambios, así que compilan. Y la última función tiene que seguir llamándose `RespuestaClimaDto.aEntidad()`, porque `ClimaRepository` la llama así.

Qué pasa cuando buscas "ditto" (todo junto)

```
1. Escribes "ditto" y pulsas Buscar
2. ClimaRepository llama:  api.obtenerPronostico("pokeapi", "ditto", 3)
3. WeatherApiService arma: @GET("pokemon/{nombre}") + @Path → pokemon/ditto
                           @Header key=pokeapi, days=3 → van dentro (PokeAPI los ignora)
4. ApiClient le pega la base: https://pokeapi.co/api/v2/pokemon/ditto
5. PokeAPI responde el JSON: { "id": 132, "name": "ditto", "height": 3, ... }
6. Gson llena el DTO:       RespuestaClimaDto(id=132, nombre="ditto", altura=3, peso=40, ...)
7. El mapper acomoda:       Ubicacion(name="ditto", region="Pokémon #132", ...)
                            ClimaActual(temp_c=0.3, feelslike_c=4.0, icon=imagen, ...)
8. La pantalla muestra:     ditto · Pokémon #132 · 0,3°C · [imagen] · Sensación térmica 4 °C
```

Cosas que vas a ver y por qué son normales

Lo que ves	Por qué
Al abrir la app: "No pudimos completar la consulta"	El ViewModel busca "La Paz, Bolivia" al arrancar, y eso no es un Pokémon. Borra el buscador y escribe `ditto`
`Ditto` con mayúscula da HTTP 404	PokeAPI solo acepta nombres en minúsculas
Un nombre que no existe da "HTTP 404"	`ClimaRepository` solo conoce los códigos de error de WeatherAPI
Siempre muestra La Paz	El modo demo está encendido: apágalo
Las etiquetas dicen "Sensación térmica", "Humedad", "Datos: WeatherAPI.com"	Son textos de la pantalla, que no tocamos
Los tests de `ClimaRepositoryTest` no compilan	Esperan el JSON y los parámetros de WeatherAPI. No afectan a la app

Revisa esto en tu código

En `ClimaMapper.kt`, línea 5, quedó un `1` suelto debajo de los imports. Bórralo: impide que la app compile.

Resumen: el porqué de todo en 5 líneas

1. **apikey.properties**: el código exige una clave, así que le damos una cualquiera.
2. **ApiClient**: otro servidor, así que otra URL base.
3. **WeatherApiService**: otra ruta (`@GET`), el nombre va dentro de la URL (`@Path`), y PokeAPI rechaza parámetros extra (`@Header` y borrar los demás).
4. **ClimaDtos**: el JSON tiene otra forma, así que el DTO copia la nueva forma.
5. **ClimaMapper**: la pantalla no cambia, así que acomodamos los datos del Pokémon en las mismas casillas.